

Sequential Information Acquisition and Optimal Search

Nawaaz Khalfan¹ Rakesh Vohra²

¹Monash University

²University of Pennsylvania

October 3, 2024

Allocating without transfers

A principal has something of value to an agent.

Information relevant to the principal for allocation is known to the agent and not the principal.

The principal has **no transfers** to discipline the allocation.

Allocating without transfers

A principal has something of value to an agent.

Information relevant to the principal for allocation is known to the agent and not the principal.

The principal has **no transfers** to discipline the allocation.

Examples

- employers **hiring** from a pool of candidates
- governments **funding** selected municipalities
- trade commissions **allowing** merger requests

Instruments

Other papers,

Evidence: the agent is able to report their private information, and the principal can verify this report for *free*.

Verification: the agent is able to report their private information, and the principal can verify this report for a *cost*.

Instruments

Other papers,

Evidence: the agent is able to report their private information, and the principal can verify this report for *free*.

Verification: the agent is able to report their private information, and the principal can verify this report for a *cost*.

This paper,

Information Acquisition: the agent is able to report their private information, and the principal can *acquire additional information* for a *cost*.

→ Call this instrument *inspection* and think of the agents information as *noisy*.

Inspection

A core economic activity

- employers **interview** candidates
- governments **appraise** municipalities
- trade commissions **assess** merger requests

Inspection

A core economic activity

- employers **interview** candidates
- governments **appraise** municipalities
- trade commissions **assess** merger requests



Inspection

A core economic activity

- employers **interview** candidates
- governments **appraise** municipalities
- trade commissions **assess** merger requests



Inspection

A core economic activity

- employers **interview** candidates
- governments **appraise** municipalities
- trade commissions **assess** merger requests



Why **inspect**?

1. discovery or *information acquisition*
2. verification or *screening*

Today's setup

Let each agents true value to the principal be binary: high, h , and low, ℓ .

The likelihood agent i is of value h is $p_i \in [0, 1]$, and is the agents **private information** or *type*. Let π_i be the prior likelihood that an agent is of type p_i .

Today's setup

Let each agents true value to the principal be binary: high, h , and low, ℓ .

The likelihood agent i is of value h is $p_i \in [0, 1]$, and is the agents **private information** or *type*. Let π_i be the prior likelihood that an agent is of type p_i .

The principal receives a reward 1 from allocating to a high agent, -1 from allocating to a low agent, and 0 from withholding allocation.

An agent receives a unit reward if allocated to, and 0 otherwise.

Today's setup

Let each agents true value to the principal be binary: high, h , and low, ℓ .

The likelihood agent i is of value h is $p_i \in [0, 1]$, and is the agents **private information** or *type*. Let π_i be the prior likelihood that an agent is of type p_i .

The principal receives a reward 1 from allocating to a high agent, -1 from allocating to a low agent, and 0 from withholding allocation.

An agent receives a unit reward if allocated to, and 0 otherwise.

For a **cost** $c > 0$, the principal can inspect an agent and discover their true value.

The principal can elicit reports, commit to an inspection and allocation mechanism, but cannot use **transfers**.

Mechanism space and objective

Suppose the principal has inspected a set of agents S and has realized a vector of rewards x_S . At this point, she has the following choices:

- inspect $i \notin S$, realise x_i and re-initialise with the set $S \cup \{i\}$
- stop and allocate to $j \in S \cup \{0\}$

Mechanism space and objective

Suppose the principal has inspected a set of agents S and has realized a vector of rewards x_S . At this point, she has the following choices:

- inspect $i \notin S$, realise x_i and re-initialise with the set $S \cup \{i\}$
- stop and allocate to $j \in S \cup \{0\}$

The principal ultimately receives her highest found reward, net of any inspection costs, making her stopping value:

$$v(S, x_S) = \max_{i \in S \cup \{0\}} x_i - \sum_{j \in S} c_j$$

The objective is to select a sequential, *nonanticipative* inspection and allocation policy that maximizes her expected stopping value.

Benchmarks

Two benchmark cases to be aware of:

Weitzman (1979) or *Pandora's problem*: if types are known to the principal ex ante this is an instance of assign each agent an index, inspect agents sequentially from high index to low index until the value of keeping the best found agent exceeds the next highest index.

Ben-Porath, Dekel and Lipman (2014): If agents are perfectly informed then *Verify* the agent with the highest reported type and allocate only to them if they match that report[†]

Forced inspection

Note that the principal **must** inspect (or search) the agent if they are to allocate to them.

There are a few reasons why we are comfortable studying this problem:

1. Institutional requirements like *due diligence* are common.
2. Pandora's problem is tractable and our construction relies on the assumption. For difficulties, see Doval (2018).
3. In *Optimal Allocation with Noisy Inspection*, allocation without inspection never occurs at the top, and is only used at the bottom to save costs.

Solution for one agent

Full information benchmark:

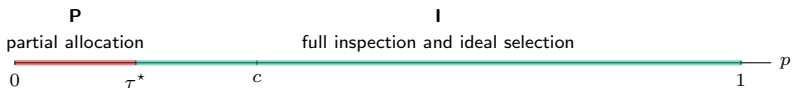


Solution for one agent

Full information benchmark:



Optimal **separating** mechanism:



Solution for multiple agents

Full information benchmark:



Solution for multiple agents

Full information benchmark:



Optimal **separating** mechanism:



Technical innovations

To show this result we:

1. recast Pandora's problem as a dynamic scheduling problem, and
2. show there are necessary and sufficient restrictions on interim allocations that guarantee ex-post implementation.

The first is due to a characterisation by Bertsimas and Niño-Mora (1996), and the second is a similar treatment as Che, Kim and Mierendorff (2013) but dates back to Hassin (1982).

Both however are in the spirit of the reduced form approach e.g. Border (1991).

Pandora's problem



Pandora's problem

A searcher, *Pandora*, can select one option, a *box*, from an available set.



Pandora's problem

A searcher, *Pandora*, can select one option, a *box*, from an available set. Selecting a box gives her an unknown reward drawn from a known distribution. For a cost, she can investigate, *open*, the box to discover its true reward.



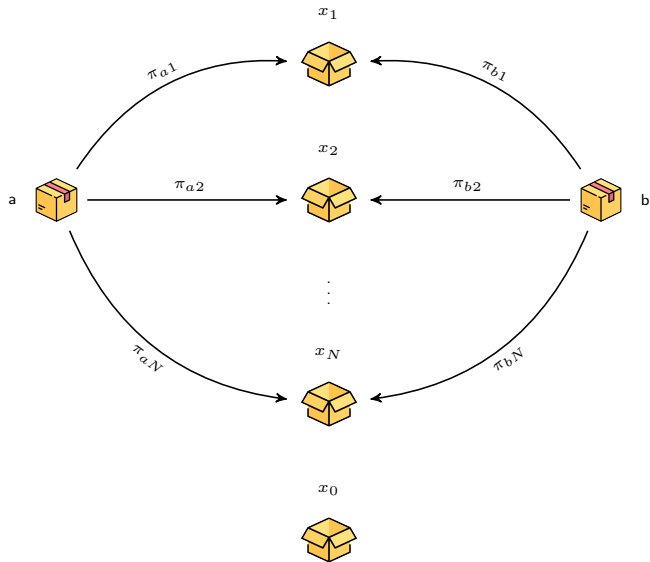
Pandora's problem

A searcher, *Pandora*, can select one option, a *box*, from an available set. Selecting a box gives her an unknown reward drawn from a known distribution. For a cost, she can investigate, *open*, the box to discover its true reward.



Weitzman (1979) showed that if Pandora must open a box before being allowed to select it, then the optimal *ordering* and *stopping rule* is *indexable*.

2 box, N prize illustration



Pandora's rule

For each box i , let its *index*, z_i , be the solution to the following:

$$c_i = \sum_{x_n > z_i} \pi_{in}(x_n - z_i)$$

Pandora's rule

For each box i , let its *index*, z_i , be the solution to the following:

$$c_i = \sum_{x_n > z_i} \pi_{in}(x_n - z_i)$$

Weitzman (1979) shows that Pandora's optimal policy is described by the following algorithm:

1. Set the *reserve value* as the value of the outside option, x_0 .
2. If the highest index among the set of unopened boxes exceeds the reserve value, open that box. Otherwise, stop and take the reserve value.
3. If the realized prize exceeds the reserve value, replace the reserve value with this new prize and return to step 2.

A dynamic scheduling problem

We can re-imagine Pandora's problem as an *infinite horizon dynamic scheduling problem* by following the lead of Bertsimas and Niño-Mora (1996).

A *class* is a particular state, and if a *job* is in class $j \in \mathcal{J}$ and is *served*, then the scheduler receives a *service reward* r_j and the job transitions to a new class $k \in \mathcal{J}$ with *transition probability* p_{jk} .

A dynamic scheduling problem

We can re-imagine Pandora's problem as an *infinite horizon dynamic scheduling problem* by following the lead of Bertsimas and Niño-Mora (1996).

A *class* is a particular state, and if a *job* is in class $j \in \mathcal{J}$ and is *served*, then the scheduler receives a *service reward* r_j and the job transitions to a new class $k \in \mathcal{J}$ with *transition probability* p_{jk} .

For Pandora, $j \in \Omega \cup \mathcal{N} \cup \{0\} = \mathcal{J}$, and,

A dynamic scheduling problem

We can re-imagine Pandora's problem as an *infinite horizon dynamic scheduling problem* by following the lead of Bertsimas and Niño-Mora (1996).

A *class* is a particular state, and if a *job* is in class $j \in \mathcal{J}$ and is *served*, then the scheduler receives a *service reward* r_j and the job transitions to a new class $k \in \mathcal{J}$ with *transition probability* p_{jk} .

For Pandora, $j \in \Omega \cup \mathcal{N} \cup \{0\} = \mathcal{J}$, and,

- if $j \in \Omega$: $r_j = -c_j$ and $p_{jn} = \pi_{jn}$ for all $n \in \mathcal{N}$,

A dynamic scheduling problem

We can re-imagine Pandora's problem as an *infinite horizon dynamic scheduling problem* by following the lead of Bertsimas and Niño-Mora (1996).

A *class* is a particular state, and if a *job* is in class $j \in \mathcal{J}$ and is *served*, then the scheduler receives a *service reward* r_j and the job transitions to a new class $k \in \mathcal{J}$ with *transition probability* p_{jk} .

For Pandora, $j \in \Omega \cup \mathcal{N} \cup \{0\} = \mathcal{J}$, and,

- if $j \in \Omega$: $r_j = -c_j$ and $p_{jn} = \pi_{jn}$ for all $n \in \mathcal{N}$,
- if $j \in \mathcal{N}$: $r_j = x_j(1 - \beta)$ and $p_{jj} = 1$, and

A dynamic scheduling problem

We can re-imagine Pandora's problem as an *infinite horizon dynamic scheduling problem* by following the lead of Bertsimas and Niño-Mora (1996).

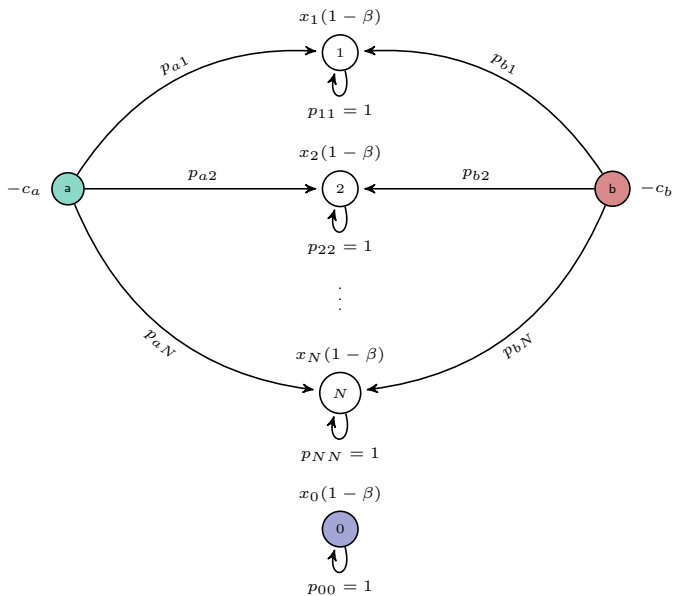
A *class* is a particular state, and if a *job* is in class $j \in \mathcal{J}$ and is *served*, then the scheduler receives a *service reward* r_j and the job transitions to a new class $k \in \mathcal{J}$ with *transition probability* p_{jk} .

For Pandora, $j \in \Omega \cup \mathcal{N} \cup \{0\} = \mathcal{J}$, and,

- if $j \in \Omega$: $r_j = -c_j$ and $p_{jn} = \pi_{jn}$ for all $n \in \mathcal{N}$,
- if $j \in \mathcal{N}$: $r_j = x_j(1 - \beta)$ and $p_{jj} = 1$, and
- if $j = 0$: $r_j = x_0(1 - \beta)$ and $p_{00} = 1$.

with $\beta \in (0, 1)$ the associated discount factor over the infinite time horizon.

2 box, N prize scheduling



Service time formulation

If $\mathcal{U}$ be the set of all admissible *policies*, then the value of the optimal dynamic schedule, $\mathcal{Z}$, is:

$$\mathcal{Z} = \max \left\{ \mathbb{E}_u \left[\sum_{t=0}^{\infty} \sum_{j \in \mathcal{J}} r_j \mathbb{1}_j(t) \beta^t \right] \mid u \in \mathcal{U} \right\} \quad (\text{DSP})$$

Service time formulation

If $\mathcal{U}$ be the set of all admissible *policies*, then the value of the optimal dynamic schedule, $\mathcal{Z}$, is:

$$\mathcal{Z} = \max \left\{ \mathbb{E}_u \left[\sum_{t=0}^{\infty} \sum_{j \in \mathcal{J}} r_j \mathbb{1}_j(t) \beta^t \right] \mid u \in \mathcal{U} \right\} \quad (\text{DSP})$$

Define λ_j as the expected discounted number of times a job in class j , and Λ the space where these service times live, then we can outline an equivalent mathematical program:

$$\mathcal{Z} = \max \sum_{j \in \mathcal{J}} r_j \lambda_j \quad \text{s.t.} \quad (\lambda_j)_{\mathcal{J}} \in \Lambda \quad (\text{MP})$$

It turns out that Λ is fully characterized by a (relatively) small number of conservation laws ...

Pandora's linear program

... and maximising a linear objective over Λ admits an indexable and decomposable solution!

Theorem 1 (Bertsimas and Niño-Mora)

The performance region, Λ , is an extended polymatroid, and for each class $j \in \mathcal{J}$ there exists indices, z , depending only on characteristics of that class, such that an optimal policy is to schedule a job with the largest current index.

Pandora's linear program

... and maximising a linear objective over Λ admits an indexable and decomposable solution!

Theorem 1 (Bertsimas and Niño-Mora)

The performance region, Λ , is an extended polymatroid, and for each class $j \in \mathcal{J}$ there exists indices, z , depending only on characteristics of that class, such that an optimal policy is to schedule a job with the largest current index.

In its generality (Bertsimas and Niño-Mora, 1996), this reproves the celebrated result of Gittins and Jones (1974).

A primer on polymatroids

For $\mathcal{J}$, a finite set, and $f : 2^{\mathcal{J}} \rightarrow \mathbb{R}_+$, a non-decreasing submodular function, a **polymatroid**, $\mathcal{P}$, is defined as the following polytope:

$$\mathcal{P} = \{x \in \mathbb{R}_+^{\mathcal{J}} \mid \sum_{j \in S} x_j \leq f(S) \ \forall S \subseteq \mathcal{J}\}$$

A primer on polymatroids

For $\mathcal{J}$, a finite set, and $f : 2^{\mathcal{J}} \rightarrow \mathbb{R}_+$, a non-decreasing submodular function, a **polymatroid**, $\mathcal{P}$, is defined as the following polytope:

$$\mathcal{P} = \{x \in \mathbb{R}_+^{\mathcal{J}} \mid \sum_{j \in S} x_j \leq f(S) \ \forall S \subseteq \mathcal{J}\}$$

Intuitively, maximising a linear objective over a polymatroid is achieved by running a *greedy algorithm*. For example, see Border (1991).

A primer on polymatroids

For $\mathcal{J}$, a finite set, and $f : 2^{\mathcal{J}} \rightarrow \mathbb{R}_+$, a non-decreasing submodular function, a **polymatroid**, $\mathcal{P}$, is defined as the following polytope:

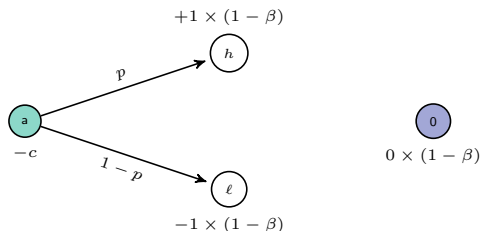
$$\mathcal{P} = \{x \in \mathbb{R}_+^{\mathcal{J}} \mid \sum_{j \in S} x_j \leq f(S) \ \forall S \subseteq \mathcal{J}\}$$

Intuitively, maximising a linear objective over a polymatroid is achieved by running a *greedy algorithm*. For example, see Border (1991).

An **extended polymatroid** is a polymatroid with coefficients, $a_{j,S}$, that do not break this greedy characterisation:

$$\mathcal{P}_\varepsilon = \{x \in \mathbb{R}_+^{\mathcal{J}} \mid \sum_{j \in S} a_{j,S} \cdot x_j \leq f(S) \ \forall S \subseteq \mathcal{J}\}$$

1 box, 2 prize scheduling problem



$$\begin{aligned}
 V = \max_{\lambda \geq 0} \quad & -c\lambda_a \quad + \quad (1 - \beta)\lambda_h \quad - \quad (1 - \beta)\lambda_\ell \\
 \text{s.t.} \quad & \lambda_a \quad + \quad \lambda_h \quad + \quad \lambda_\ell \quad + \quad \lambda_0 \quad = \quad \frac{1}{1 - \beta} \\
 & \lambda_a \quad \leq \quad 1 \\
 & -\frac{p\beta}{1 - \beta}\lambda_a \quad + \quad \lambda_h \quad \leq \quad 0 \\
 & -\frac{(1-p)\beta}{1 - \beta}\lambda_a \quad \quad \quad + \quad \lambda_\ell \quad \leq \quad 0
 \end{aligned}$$

Full information

Suppose Pandora knows p . Then, following our formulation, Pandora's **interim first best value** is given by:

$$\begin{aligned}
 \bar{v}(p) = \max_{\lambda \geq 0} \quad & -c\lambda_a \quad + \quad (1-\beta)\lambda_h \quad - \quad (1-\beta)\lambda_\ell \\
 \text{s.t.} \quad & \lambda_a \quad + \quad \lambda_h \quad + \quad \lambda_\ell \quad + \quad \lambda_0 \quad = \quad \frac{1}{1-\beta} \\
 & \lambda_a \quad \leq \quad 1 \\
 & -\frac{p\beta}{1-\beta}\lambda_a \quad + \quad \lambda_h \quad \leq \quad 0 \\
 & -\frac{(1-p)\beta}{1-\beta}\lambda_a \quad \quad \quad + \quad \lambda_\ell \quad \leq \quad 0
 \end{aligned}$$

Full information

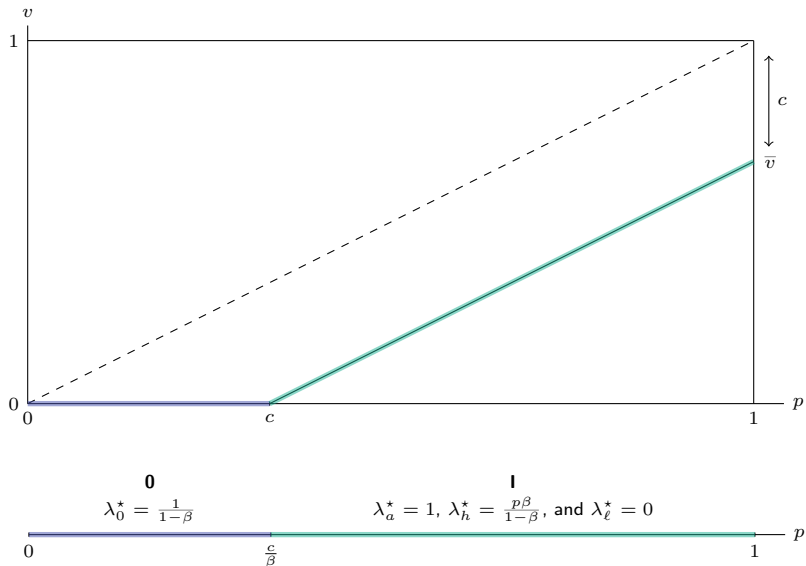
Suppose Pandora knows p . Then, following our formulation, Pandora's **interim first best value** is given by:

$$\begin{aligned} \bar{v}(p) = \max_{\lambda \geq 0} \quad & -c\lambda_a \quad + \quad (1-\beta)\lambda_h \quad - \quad (1-\beta)\lambda_\ell \\ \text{s.t.} \quad & \lambda_a \quad + \quad \lambda_h \quad + \quad \lambda_\ell \quad + \quad \lambda_0 \quad = \quad \frac{1}{1-\beta} \\ & \lambda_a \quad \leq \quad 1 \\ & -\frac{p\beta}{1-\beta}\lambda_a \quad + \quad \lambda_h \quad \leq \quad 0 \\ & -\frac{(1-p)\beta}{1-\beta}\lambda_a \quad + \quad \lambda_\ell \quad \leq \quad 0 \end{aligned}$$

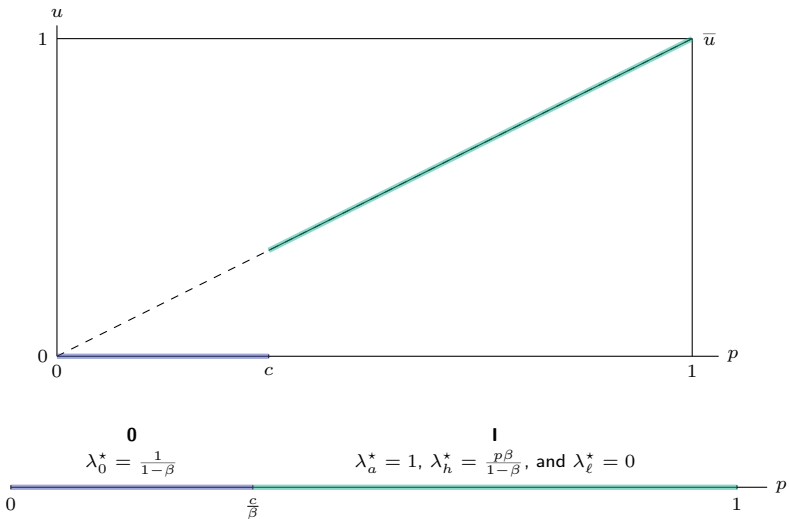
A solution to this problem is λ^* , where:

- if $p\beta - c \geq 0$, then $\lambda_a^* = 1$, $\lambda_h^* = \frac{p\beta}{1-\beta}$, and $\lambda_\ell^* = 0$, and
- if $p\beta - c < 0$, then $\lambda_0^* = \frac{1}{1-\beta}$.

First best value and policy



First best payoff is not incentive compatible



Incentive compatibility

The allocations, $\lambda_h(q)$ and $\lambda_\ell(q)$, are only ever given after h or ℓ has been observed, and so, the deviation payoff for a type p reporting q is:

$$u(q|p) = \frac{p}{q} \lambda_h(q) + \frac{1-p}{1-q} \lambda_\ell(q)$$

Incentive compatibility

The allocations, $\lambda_h(q)$ and $\lambda_\ell(q)$, are only ever given after h or ℓ has been observed, and so, the deviation payoff for a type p reporting q is:

$$u(q|p) = \frac{p}{q} \lambda_h(q) + \frac{1-p}{1-q} \lambda_\ell(q)$$

Collecting terms gives us a familiar form:

$$u(q|p) = \underbrace{\frac{\lambda_\ell(q)}{1-q}}_{\text{guarantee}} + \underbrace{p}_{\text{true type}} \cdot \underbrace{\left[\frac{\lambda_h(q)}{q} - \frac{\lambda_\ell(q)}{1-q} \right]}_{\text{allocation differential}}$$

Incentive compatibility

The allocations, $\lambda_h(q)$ and $\lambda_\ell(q)$, are only ever given after h or ℓ has been observed, and so, the deviation payoff for a type p reporting q is:

$$u(q|p) = \frac{p}{q} \lambda_h(q) + \frac{1-p}{1-q} \lambda_\ell(q)$$

Collecting terms gives us a familiar form:

$$u(q|p) = \underbrace{\frac{\lambda_\ell(q)}{1-q}}_{\text{guarantee}} + \underbrace{p}_{\text{true type}} \cdot \underbrace{\left[\frac{\lambda_h(q)}{q} - \frac{\lambda_\ell(q)}{1-q} \right]}_{\text{allocation differential}}$$

Let the **guarantee** for report q be denoted by $y_\ell(q)$ and the **differential** for q be denoted by $\Delta(q)$. The **incentive compatibility constraints** can be written as:

$$IC_{pq} : u(p|p) = y_\ell(p) + p\Delta(p) \geq y_\ell(q) + p\Delta(q) = u(q|p) \quad \forall p, q \in \mathcal{P}$$

Relaxation

Suppose we have a grid of n types, $\mathcal{P}$, ordered such that $p_i > p_j$ if $i < j$.

Relaxation

Suppose we have a grid of n types, $\mathcal{P}$, ordered such that $p_i > p_j$ if $i < j$.

As is common, many IC constraints are redundant. Here, in any second-best policy, the *local upward incentive compatibility constraints* must bind and the differential must be *monotone*.

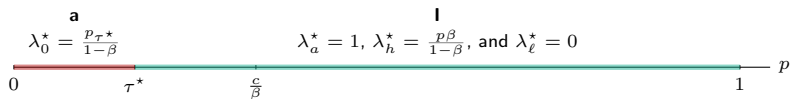
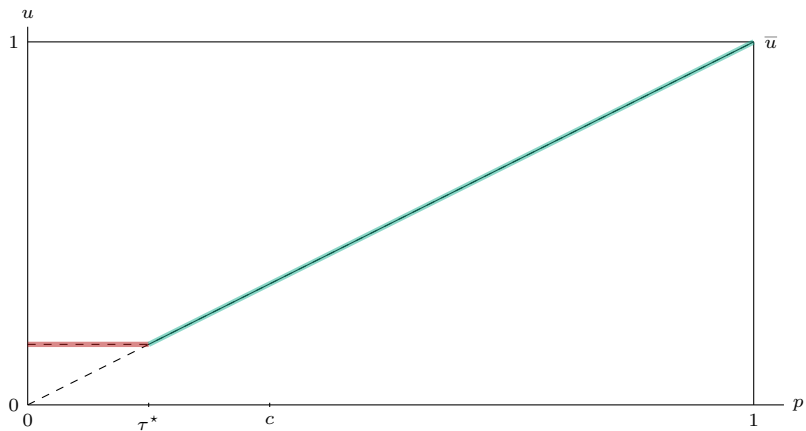
That is, for all $i < n$,

$$\lambda_h(i) + \lambda_\ell(i) = \frac{p_i}{p_{i+1}} \lambda_h(i+1) + \frac{1-p_i}{1-p_{i+1}} \lambda_\ell(i+1)$$

and

$$\Delta(i) \leq \Delta(i+1)$$

Second-best payoff and policy



Multiple agents

Now suppose there are n agents, indexed $i \in \{1, \dots, n\} =: N$ and let T be the set of possible types of an agent.

Denote by $\lambda_{ij}(t)$ the allocation to agent $i \in N$ at state $j \in \Omega$ at the profile $t = (t_1, \dots, t_n)$ of types. In our case, there is a special state, denoted '0' and the probability of transitioning from state 0 to state $j \in \Omega \setminus \{0\}$ for agent i of type t_i is denoted $p(j|t_i)$.

Denote the set of all profiles of types by T^n , and the likelihood of any particular type as $\pi(t)$. We will assume that types are independently drawn so that: $\pi(t) = \pi(t_i)\pi(t_{-i})$.

The reduced form

Let Q denote the interim allocation variables so that:

$$\pi(t_i)Q_{ij}(t_i) = \sum_{t_{-i}} \pi(t_i)\pi(t_{-i})\lambda_{ij}(t_i, t_{-i})$$

The reduced form

Let Q denote the interim allocation variables so that:

$$\pi(t_i)Q_{ij}(t_i) = \sum_{t_{-i}} \pi(t_i)\pi(t_{-i})\lambda_{ij}(t_i, t_{-i})$$

The objective function can now be rewritten as:

$$\begin{aligned} V &= \sum_{t \in T^n} \pi(t) \sum_{i \in N} \sum_{j \in \Omega} r_j \lambda_{ij}(t) \\ \Rightarrow V &= \sum_{i \in N} \sum_{t_i \in T} \pi(t_i) \sum_{j \in \Omega} r_j Q_{ij}(t_i) \end{aligned}$$

This says that any two policies that have the same average, or *interim*, service times, are equivalent in terms of their value to the principal.

Transport formulation

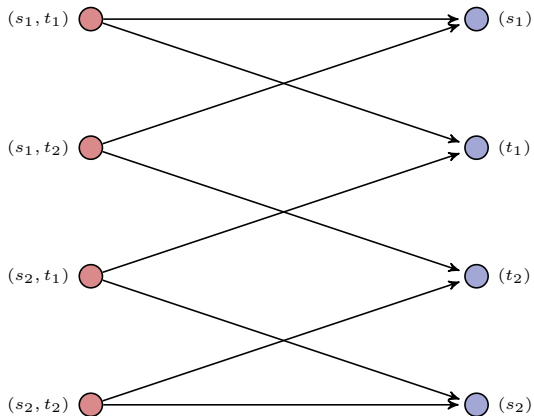
Let $\Delta := N \times T \times \Omega$ be the set of interim states or *demand nodes* and $S := T^n$ be the set of ex post states or *supply nodes*.

Let $\lambda(s) = (\lambda(s, d))_{d \in N(s)}$ be the ex post allocation or *supply* at node $s \in S$, and $Q(d)$ be the interim allocation or *demand* at node $d \in \Delta$.

The total *supply* from node s to a collection of *demand* nodes $D \subseteq N(s)$ is, by our first theorem, bounded by a submodular function g :

$$\sum_{d \in D} \lambda(s, d) \leq g(D|s)$$

A 2 agent, 2 type transport problem



A general solution

Those familiar with transport problems - e.g. Gale and Shapley (1962) - will then recognise the type of result we get for this problem.

Theorem 2 (Hassin, Che, Kim and Mierendorff)

There exists a λ such that,

- $\sum_{s \in N(d)} \pi(s, d) \lambda(s, d) = Q(d) \quad \forall d \in \Delta, \text{ and}$
- $0 \leq \sum_{d \in D} x(s, d) \leq g(D|s) \quad \forall D \subseteq N(s), s \in S$

if and only if,

$$0 \leq \sum_{d \in D} \pi(d) Q(d) \leq \sum_{s \in N(D)} \pi(s) g(D \cap N(s)|s) \quad \forall D \subseteq \Delta$$

Note that this essentially reproves Border (1991).

Optimal multiple agent mechanism

This means our proof for the one agent case carries through, so long as we respect these new upper bounds on interim allocations.

Optimal multiple agent mechanism

This means our proof for the one agent case carries through, so long as we respect these new upper bounds on interim allocations.

Theorem 3 (Threshold Rule)

The optimal separating mechanism sets a threshold τ^ , such that for each agent i ,*

- if $p \geq \tau$, $Q_{i0}(p) = Q_{i0}(\bar{p})$, $Q_{i\ell}(p) = 0$ and $Q_{ih}(p) = \frac{\beta p}{1-\beta} Q_{i0}(p)$, and
- if $p < \tau$, $Q_{i0}(p) = \frac{\beta \tau}{1-\beta} Q_{i0}(\bar{p})$, $Q_{i\ell}(p) = \frac{\beta(1-p)}{1-\beta} Q_{i0}(p)$ and $Q_{ih}(p) = \frac{\beta p}{1-\beta} Q_{i0}(p)$,

where $Q_{i0}(\bar{p})$ is set maximally.

Optimal *separating* mechanism:



Noisy inspection

Optimally **informed** search balances *discovery* and *verification*, at most only partially exploiting private information to guide search.

To show this we:

1. recast Pandora's problem as a dynamic scheduling problem, and
2. show there are necessary and sufficient restrictions on interim allocations that guarantee ex-post implementation.

Limitations?

- Forced inspection → seems innocuous
- Binary prizes → need *some* flattening e.g. MLRP
- Comparative statics → missing from this treatment

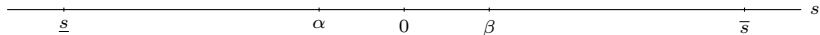
References

- Ben-Porath, Elchanan, Eddie Dekel and Barton L. Lipman. 2014. "Optimal Allocation with Costly Verification." *American Economic Review* 104(12):3779–3813.
- Bertsimas, Dimitris and José Niño-Mora. 1996. "Conservation Laws, Extended Polymatroids and Multiarmed Bandit Problems; A Polyhedral Approach to Indexable Systems." *Mathematics of Operations Research* 21(2):257–306.
- Border, Kim C. 1991. "Implementation of Reduced Form Auctions: A Geometric Approach." *Econometrica* 59(4):1175–1187.
- Che, Yeon-Koo, Jinwoo Kim and Konrad Mierendorff. 2013. "Generalized Reduced-Form Auctions: A Network-Flow Approach." *Econometrica* 81(6):2487–2520.
- Doval, Laura. 2018. "Whether or not to open Pandora's box." *Journal of Economic Theory* 175:127–158.
- Gale, D. and L. S. Shapley. 1962. "College Admissions and the Stability of Marriage." *The American Mathematical Monthly* 69(1):9–15.
- Gittins, J. C. and D. M. Jones. 1974. *A Dynamic Allocation Index for the Sequential Design of Experiments*. Amsterdam: North-Holland pp. 241–266.
- Hassin, Refael. 1982. "Minimum cost flow with set-constraints." *Networks* 12:1–21.
- Khalfan, Nawaaz. 2023. "Optimal Allocation with Noisy Inspection."
- Weitzman, M. 1979. "Optimal Search for the Best Alternative." *Econometrica* 47(3):641–654.

Let r be the principal's **reward**, and s be the agent's **type**, sorted and labelled by the expected value of the reward.

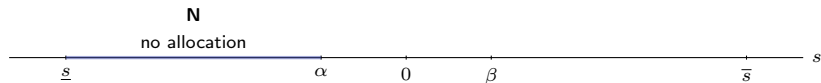
Let r be the principal's **reward**, and s be the agent's **type**, sorted and labelled by the expected value of the reward.

Symmetric information benchmark:



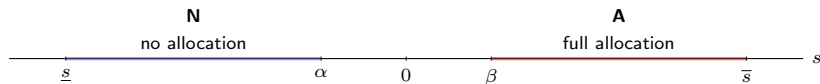
Let r be the principal's **reward**, and s be the agent's **type**, sorted and labelled by the expected value of the reward.

Symmetric information benchmark:



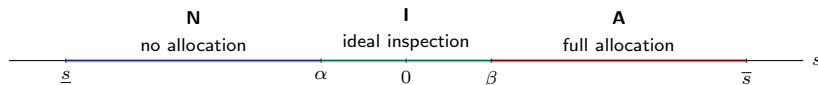
Let r be the principal's **reward**, and s be the agent's **type**, sorted and labelled by the expected value of the reward.

Symmetric information benchmark:



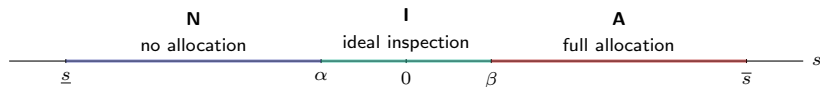
Let r be the principal's **reward**, and s be the agent's **type**, sorted and labelled by the expected value of the reward.

Symmetric information benchmark:

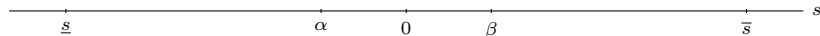


Let r be the principal's **reward**, and s be the agent's **type**, sorted and labelled by the expected value of the reward.

Symmetric information benchmark:

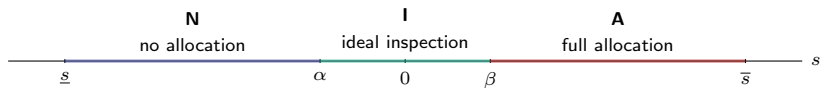


Optimal mechanism:

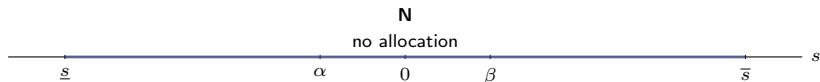


Let r be the principal's **reward**, and s be the agent's **type**, sorted and labelled by the expected value of the reward.

Symmetric information benchmark:

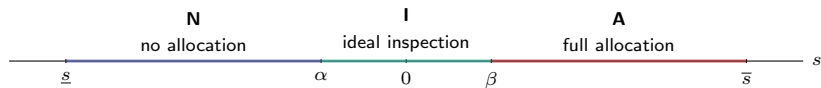


Optimal pooling mechanism:

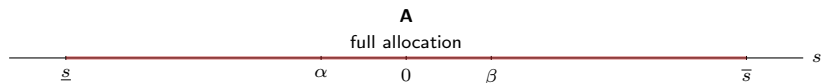


Let r be the principal's **reward**, and s be the agent's **type**, sorted and labelled by the expected value of the reward.

Symmetric information benchmark:

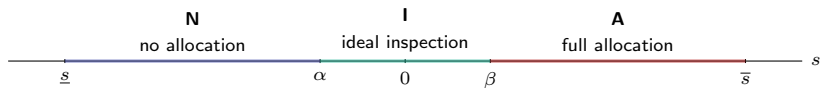


Optimal pooling mechanism:

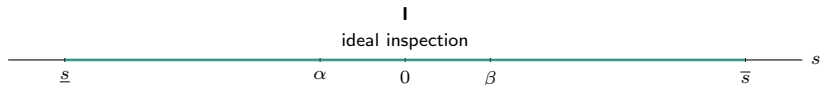


Let r be the principal's **reward**, and s be the agent's **type**, sorted and labelled by the expected value of the reward.

Symmetric information benchmark:

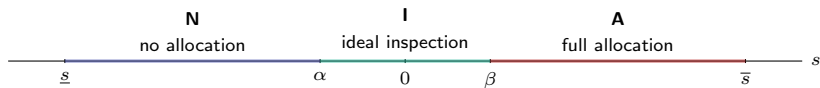


Optimal pooling mechanism:

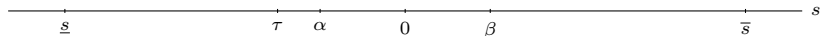


Let r be the principal's **reward**, and s be the agent's **type**, sorted and labelled by the expected value of the reward.

Symmetric information benchmark:

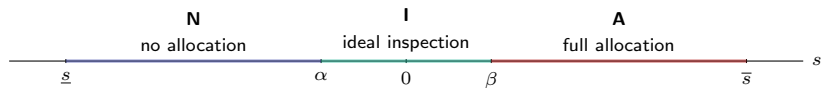


Optimal **separating** mechanism:

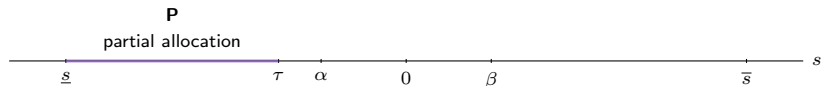


Let r be the principal's **reward**, and s be the agent's **type**, sorted and labelled by the expected value of the reward.

Symmetric information benchmark:

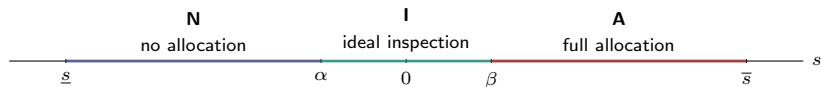


Optimal separating mechanism:



Let r be the principal's **reward**, and s be the agent's **type**, sorted and labelled by the expected value of the reward.

Symmetric information benchmark:



Optimal separating mechanism:

